

Day 7 - Test-Driven Development

Next week you build a game **solo** in seven days. Tests are what let you refactor fast without breaking yesterday's work, and they are your safety net during the jam. Today you learn to write the test *first*, watch it fail, make it pass, and clean up: the discipline that keeps a solo project from collapsing under its own weight.

Learning objectives

By the end of today you can:

- [ ] Run the red-green-refactor cycle deliberately.
- [ ] Write a test first, then the code that satisfies it.
- [ ] Set up and use a unit-testing framework in C++ (the gauntlet uses Catch2).
- [ ] Use test fixtures to share setup across tests.
- [ ] Use mocks/fakes to isolate the unit under test from its collaborators.

Concepts

> **Framework for the gauntlet: Catch2 (v3, amalgamated).** No install and no CMake: two files, `catch_amalgamated.hpp` and `catch_amalgamated.cpp`, dropped next to your code. The `.hpp` is what your tests include; the `.cpp` is the framework itself, compiled alongside them. All snippets below use it.

Red-green-refactor

The loop: write one small failing test (**red**), write the least code that makes it pass (**green**), then clean up with the test guarding you (**refactor**). A healthy cycle is minutes long: red in one, green in a few. If green takes an hour, the step was too big.

One cycle on the `Inventory` the exercises use:

1. **Red:** write `TEST_CASE("adding an item raises the count")`: construct an `Inventory`, `add("potion")`, `CHECK(inv.count() == 1)`. It doesn't even compile, since `Inventory` doesn't exist yet. A compile error is a valid red.
2. **Green:** the least code that passes. `add` pushes onto a `std::vector`, `count` returns its size. Run the tests, watch it pass.
3. **Refactor:** rename, extract, simplify. The suite staying green is the *proof* the cleanup broke nothing, and that proof is the whole payoff of the loop.

❌ **Common Mistake:** skipping red, writing the code first and then a test you've never seen fail. A test that can't fail guards nothing: when the behaviour later breaks, the test keeps passing and the regression ships.

✅ **Good Practice:** watch every test fail once before you make it pass. That failure is the proof the test is actually wired to the behaviour.

Tests first

Writing the test before the code forces you to use the interface before you build it, so you design the API you actually want and stop over-building features no test asks for. The test is the first *client* of your class.

```
// Writing this first tells you Inventory needs add() and count(), and
nothing more yet.
TEST_CASE("adding an item raises the count") {
    Inventory inv;
    inv.add("potion");
    CHECK(inv.count() == 1);
}
```

❌ **Common Mistake:** designing the class first, then bending the test to fit what you built. The test ends up asserting the implementation ("the vector holds one element") instead of the behaviour ("the item is retrievable"), so every refactor breaks it.

✅ **Good Practice:** write the test against the interface you *wish* you had, then implement it, and assert only what a caller can observe.

A unit-testing framework: Catch2

You write no `main()` at all: `catch_amalgamated.cpp` brings one, and it runs every test that registered itself. Tests self-register via `TEST_CASE`, so there is no list to maintain and no function to call. Assert with `CHECK` (reports and keeps going) or `REQUIRE` (aborts the case).

```
// tests.cpp: no main(), no #define. catch_amalgamated.cpp supplies the
runner
#include "catch_amalgamated.hpp"
#include "Inventory.hpp"

TEST_CASE("a new inventory starts empty") {
    Inventory inv;
    REQUIRE(inv.count() == 0);       // REQUIRE aborts the case; the line
below is meaningless if this fails
    CHECK_FALSE(inv.has("potion"));  // CHECK reports and continues
}
```

Build and run: `c++ -Wall -Wextra -Werror -std=c++17 tests.cpp Inventory.cpp catch_amalgamated.cpp -o tests && ./tests`.

That third file is ~12,000 lines, so the first build is noticeably slower than the rest of the week. It only bites once: the Makefile from Day 0 compiles it to `catch_amalgamated.o` and reuses that object on every later build, which is exactly what the `%.o: %.cpp` rule is for.

❌ **Common Mistake:** including the header but leaving `catch_amalgamated.cpp` out of the build. The compile succeeds and the *link* dies on a wall of undefined `Catch::` symbols (`Catch::makeTestInvoker`, `Catch::AssertionHandler::complete`…), which reads like a corrupt download and sends people off re-downloading a file that was never the problem.

✅ **Good Practice:** put `catch_amalgamated.cpp` in the Makefile's `SRCS` the moment you copy it in; every other file only ever includes the `.hpp`.

Test fixtures

When several tests need the same setup, share it via a fixture struct: Catch2 constructs a fresh instance per test (`TEST_CASE_METHOD`), so tests stay independent and no state leaks between them. The test body becomes a method of the struct, which is why it reaches `inv` with no qualification.

```
struct InventoryFixture {
    Inventory inv;                // Catch2 builds a FRESH instance for
every test below
    InventoryFixture() { inv.add("potion"); inv.add("sword"); }
};

TEST_CASE_METHOD(InventoryFixture, "removing drops the count") {
    inv.remove("sword");
    CHECK(inv.count() == 1);
}
// a second TEST_CASE_METHOD asserting the starting state passes even when
it
// runs AFTER the removal above. Why? proving that guarantee is ex01's job
```

❌ **Common Mistake:** sharing setup through a global or `static` object instead of fixture members. One test's mutation leaks into the next, so the suite passes in one order and fails in another, and a test that fails alone passes in the full run.

✅ **Good Practice:** put the shared setup in a fixture struct, which Catch2 rebuilds fresh for every `TEST_CASE_METHOD`. That independence is exactly what fixtures exist for.

Mocking / fakes

To test code that depends on something non-deterministic (RNG, clock, file), inject the dependency behind an interface and pass a controllable **fake** in tests. Now the test is deterministic and isolated from the real world.

```
struct RandomSource {                    // the seam: the one thing a test
must control
    virtual int next(int max)= 0;       // returns 0 .. max-1
    virtual ~RandomSource() = default;   // deleted through a base pointer:
day-3 rule
```

```
};

class Dice {
public:
    explicit Dice(RandomSource& rng) : m_rng(rng) {}   // injected: Dice
never creates its own RNG
    int roll();   // maps next(6) onto a face 1..6. which face does 0
become, and why?
private:
    RandomSource& m_rng;
};

// in tests.cpp: a FakeRandom that implements next() by returning one stored
value.
// with the rng pinned, roll()'s mapping becomes checkable with an exact
CHECK
```

❌ **Common Mistake:** testing randomness statistically ("roll 1000 times, expect each face roughly ⅙"). It's flaky by design: it fails occasionally on correct code and passes on subtly biased code, so people learn to re-run until green and stop trusting the suite.

✅ **Good Practice:** inject a fake that returns a pinned value, then assert the exact result with a single `CHECK`.

Exercises

Get `catch_amalgamated.hpp` and `catch_amalgamated.cpp` (both from the Catch2 releases page; you already downloaded them in Day 0 - Setup) and copy **both** into each exercise folder, next to your own sources. The `.cpp` goes in that folder's Makefile `SRCS`; tests build and run with one command per the setup snippet above.

ex00 - Red-green-refactor a kata

.	.	.
💢	**Difficulty**	⭐⭐
🧠	**Objectives**	*Implement a small `Inventory` kata strictly test-first with Catch2, committing at each red→green step so the cycle shows in the log.*
📁	**Turn in directory**	`ex00/`
❗	**Turn in files**	**Inventory.hpp/.cpp, tests.cpp, catch_amalgamated.hpp/.cpp, Makefile**
🛠️	**Skills required**	*Red-green-refactor, writing a failing test first, Catch2 `TEST_CASE` / `CHECK`, committing per cycle step.*
🔍	**Search keywords**	*Catch2 TEST_CASE CHECK, catch2 amalgamated build, test driven development red green refactor, test first C++*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`"catch_amalgamated.hpp"`, `<string>`, `<vector>`

- **Goal:** Implement `Inventory` **strictly test-first**: one red→green cycle per behaviour, two commits per cycle (`red: <behaviour>` while the test fails, `green: <behaviour>` once it passes), in this order:
  1. adding an item raises the count (1 assertion),
  2. `has()` finds an added item and rejects a missing one (2 assertions),
  3. removing an item drops the count and `has()` stops finding it (2 assertions),
  4. removing a missing item is a no-op: count unchanged, other items untouched (2 assertions).

  Binary name: `tests`.
- **Done when:** `git log` shows a red commit before each behaviour's green commit (four pairs), and `./tests` prints the run below: 4 test cases, 7 assertions, all passing.

Your tests drive out this interface. Every test body and every `Inventory` body is yours to write, test first:

```
// Inventory.hpp
#pragma once

#include <string>

class Inventory {
public:
    void add(const std::string& item);
```

```
    void remove(const std::string& item);   // absent item: no-op, cycle 4
proves it
    bool has(const std::string& item) const;
    int  count() const;
};
```

Output example:

```
$ ./tests
Randomness seeded to: 3567479892
===============================================================================
All tests passed (7 assertions in 4 test cases)
$ git log --oneline
1f6b2c9 green: removing a missing item is a no-op
84a0d3e red: removing a missing item is a no-op
...       (a red/green pair per behaviour)
7c3e91a green: adding an item raises the count
b2d4f08 red: adding an item raises the count
```

The seed on the first line is different every run. That's Catch2 reporting it, not your tests being random. The commit pairs are what ex00 grades: they prove each test existed and *failed* before the code that satisfies it, not just that the final suite passes.

ex01 - Fixtures

.	.	.
💢	**Difficulty**	⭐⭐
🧠	**Objectives**	*Refactor ex00's tests to share setup via a `TEST_CASE_METHOD` fixture struct, keeping every test independent of the others' order.*
📁	**Turn in directory**	`ex01/`
❗	**Turn in files**	**Inventory.hpp/.cpp (from ex00), tests.cpp, catch_amalgamated.hpp/.cpp, Makefile**
🛠️	**Skills required**	*Catch2 `TEST_CASE_METHOD`, per-test fresh setup, keeping tests order-independent.*
🔍	**Search keywords**	*Catch2 TEST_CASE_METHOD, catch2 test fixtures, shared test setup, test isolation independent, catch2 command line test spec*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`"catch_amalgamated.hpp"`, `<string>`, `<vector>`

- **Goal:** Refactor ex00's four tests to share setup through a `TEST_CASE_METHOD` fixture struct whose member `Inventory` is pre-loaded with `"potion"` and `"sword"`. Same four behaviours; only the setup moves into the fixture (the counts each test asserts shift by the two pre-loaded items). Use `"elixir"` wherever a behaviour needs an item the fixture did *not* load: the one `has()` must reject, and the one whose removal is a no-op. Tests must stay order-independent: a mutation in one must not affect another. Binary name: `tests`.
- **Done when:** The only setup left inside a test is what that test alone needs, and any single test passes in isolation, as below. Catch2 takes the test's name as a plain argument, `./tests "<name>"`.

Catch2 rebuilds the fixture fresh for every test, so a mutating test can't leak into the next. The isolation run is how you prove it:

Output example:

```
$ ./tests "removing an item drops the count"
Filters: "removing an item drops the count"
Randomness seeded to: 552344279
===============================================================================
All tests passed (2 assertions in 1 test case)
```

`Filters:` names the one case that was allowed to run, and `1 test case` confirms the other three never did. The removal test passing **in isolation** proves it never leaned on another test running first. The fixture hands every test its own fresh two-item `Inventory`.

ex02 - Mock a collaborator

.	.	.
💢	**Difficulty**	⭐⭐⭐
🧠	**Objectives**	*Extract a non-deterministic dependency (RNG) behind an interface and inject a fake in the test so `Dice::roll()` asserts an exact value on every run.*
📁	**Turn in directory**	`ex02/`
❗	**Turn in files**	**Dice.hpp, RandomSource.hpp, tests.cpp, catch_amalgamated.hpp/.cpp, Makefile**
🛠️	**Skills required**	*Dependency injection behind an interface, a hand-written fake/mock, deterministic tests, isolating the unit under test.*
🔍	**Search keywords**	*C++ mock fake dependency injection, test double, inject interface rng, deterministic unit test, seam*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`"catch_amalgamated.hpp"`, `<cstdlib>`

- **Goal:** Start from the coupled `Dice` below. Extract the randomness behind a `RandomSource` interface in `RandomSource.hpp` (`virtual int next(int max)` returning `0 .. max-1`, plus a virtual destructor), then make `Dice` take a `RandomSource&` in its constructor, mapping `next(6)` onto faces `1..6`.
  In `tests.cpp`, write a `FakeRandom` that returns one fixed value, and two test cases pinning the edges of the mapping: rng value `0` must give face `1`, rng value `5` must give face `6`. Binary name: `tests`.

  `FakeRandom::next` ignores its `max`, which is the point of a fake, and `-Wextra -Werror` rejects an unused parameter. Leave the parameter *unnamed* (`int next(int) override`): the type still matches the interface, so the override is valid, and there's no name to go unused.
- **Done when:** Both tests assert exact faces and pass on every run, with no real randomness reaching the tests, matching below.

In the starting point below, `roll()` owns its RNG, so no test can pin its result:

```
// Dice.hpp: before the refactor, untestable
class Dice {
public:
    int roll(){ return std::rand() % 6 + 1; }   // randomness baked in, so
an exact CHECK is impossible
};
```

Output example:

```
$ ./tests
Randomness seeded to: 4137208950
===============================================================================
All tests passed (2 assertions in 2 test cases)
```

Injecting the fake makes `roll()` map `rng+1` deterministically, so the assertion is exact on every run. The real generator never reaches the unit under test.

Constraints (every exercise)

- Compiles with `c++ -Wall -Wextra -Werror -std=c++17`.
- Tests run with a single documented command (`./tests`, built via the `Makefile`).
- Tests are deterministic, with no real randomness or time leaking in.
- `catch_amalgamated.cpp` is listed in the Makefile's `SRCS`, so it compiles once and links into `tests`. You write no `main()` of your own.

How you're evaluated

An evaluation will be automatically scheduled with a peer and start at 6pm. Submit all the work you have done so far even if its not 100% complete.

Turning in your work

1. Create a new repository on GitHub named `gauntlet-day-07`, and keep it **public**. A private repo is one your evaluator cannot clone, and the evaluation cannot start without the code.
2. Commit today's work there, one directory per exercise (`ex00/`, `ex01/`, ...) exactly as each turn-in table says, `Makefile` included.
3. Push before the evaluation. Whatever sits on the default branch when the evaluation starts is what gets read. Commit as you go rather than squashing everything into one commit: this day's evaluation reads the history to check the tests came first.
4. When the evaluation starts, give your evaluator the link to the repository.

AI-written code is an immediate disqualification. Look things up and ask an AI what something does, that is a tool. Pasting in code you did not write is not.